

Lab 7

2. Multithreading in Java (I)

Dr. Andreas S. Maniatis

Assistant Professor

School of Computer Science

COMP.2800 – Software Development [2026W]

Tuesday, March 10th, 2026 | | ER 3120



University of Windsor

Part I Threads



University of Windsor



Threads



● Threads

- • Threads are lightweight processes as the overhead of switching between threads is less.
- They can be easily spawned.
- The Java Virtual Machine spawns a thread when your program is run called the Main Thread.



Why do we need threads?

- To enhance parallel processing.
- To increase response to the user.
- To utilize the idle time of the CPU.
- Prioritize your work depending on priority.



Example

- Consider a simple web server.
- The web server listens for request and serves it.
- If the web server was not multithreaded, the requests processing would be in a queue, thus increasing the response time and might hang the server if there was a bad request.
- By implementing in a multithreaded environment, the web server can serve multiple request simultaneously thus improving response time.





Creating Threads



Creating threads

- In java threads can be created by extending the Thread class or implementing the Runnable Interface.
- It is more preferred to implement the Runnable Interface so that we can extend properties from other classes.
- Implement the run() method which is the starting point for thread execution.

Creating threads Extending Thread Class

```
// Java code for thread creation by extending
// the Thread class
class MultithreadingDemo extends Thread {
    public void run()
    {
        try {
            // Displaying the thread that is running
            System.out.println(
                "Thread " + Thread.currentThread().getId()
                + " is running");
        }
        catch (Exception e) {
            // Throwing an exception
            System.out.println("Exception is caught");
        }
    }
}

// Main Class
public class Multithread {
    public static void main(String[] args)
    {
        int n = 8; // Number of threads
        for (int i = 0; i < n; i++) {
            MultithreadingDemo object
                = new MultithreadingDemo();
            object.start();
        }
    }
}
```

Creating threads Implements Runnable

```
// Java code for thread creation by implementing
// the Runnable Interface
class MultithreadingDemo implements Runnable {
    public void run()
    {
        try {
            // Displaying the thread that is running
            System.out.println(
                "Thread " + Thread.currentThread().getId()
                + " is running");
        }
        catch (Exception e) {
            // Throwing an exception
            System.out.println("Exception is caught");
        }
    }
}

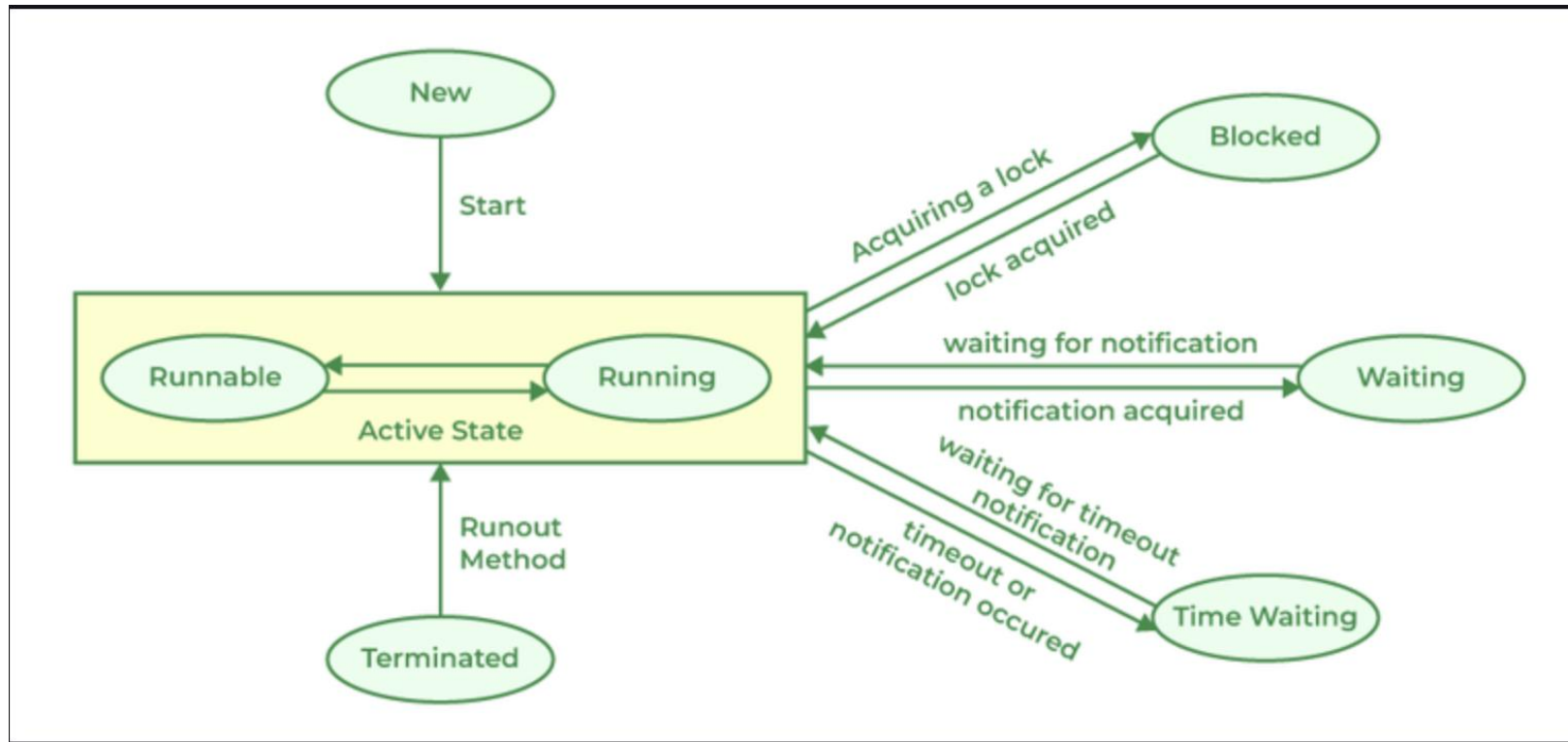
// Main Class
class Multithread {
    public static void main(String[] args)
    {
        int n = 8; // Number of threads
        for (int i = 0; i < n; i++) {
            Thread object
                = new Thread(new MultithreadingDemo());
            object.start();
        }
    }
}
```



Thread States



Thread States



Race Condition

- Definition: A race condition is a situation that occurs in concurrent programming when the behavior of a program depends on the relative timing or interleaving of multiple threads or processes accessing shared resources.

● Race Condition Demo

- • Run the code available on Brightspace.





Synchronization



● Synchronization

- Synchronization is preventing data corruption
- Synchronization allows only one thread to perform an operation on an object at a time.
- If multiple threads require an access to an object, synchronization helps in maintaining consistency.



Example Using Synchronized

```
1  class Counter {
2      private int count = 0;
3
4      public void increment() {
5          synchronized (Counter.class) {
6              count++;
7          }
8      }
9
10     public int getCount() {
11         return count;
12     }
13 }
14
15
16 public class RaceConditionExample {
17     Run | Debug
18     public static void main(String[] args) {
19         Counter counter = new Counter();
20
21         // Create multiple threads to increment the counter concurrently
22         Thread thread1 = new Thread(() -> {
23             for (int i = 0; i < 1000; i++) {
24                 counter.increment();
25             }
26         });
27
28         Thread thread2 = new Thread(() -> {
29             for (int i = 0; i < 1000; i++) {
30                 counter.increment();
31             }
32         });
33
34         // Start the threads
35         thread1.start();
36         thread2.start();
37
38         // Wait for threads to finish
39         try {
40             thread1.join();
41             thread2.join();
42         } catch (InterruptedException e) {
43             e.printStackTrace();
44         }
45
46         // Display the final count
47         System.out.println("Final count: " + counter.getCount());
48     }
49 }
```

Producer – Consumer Problem



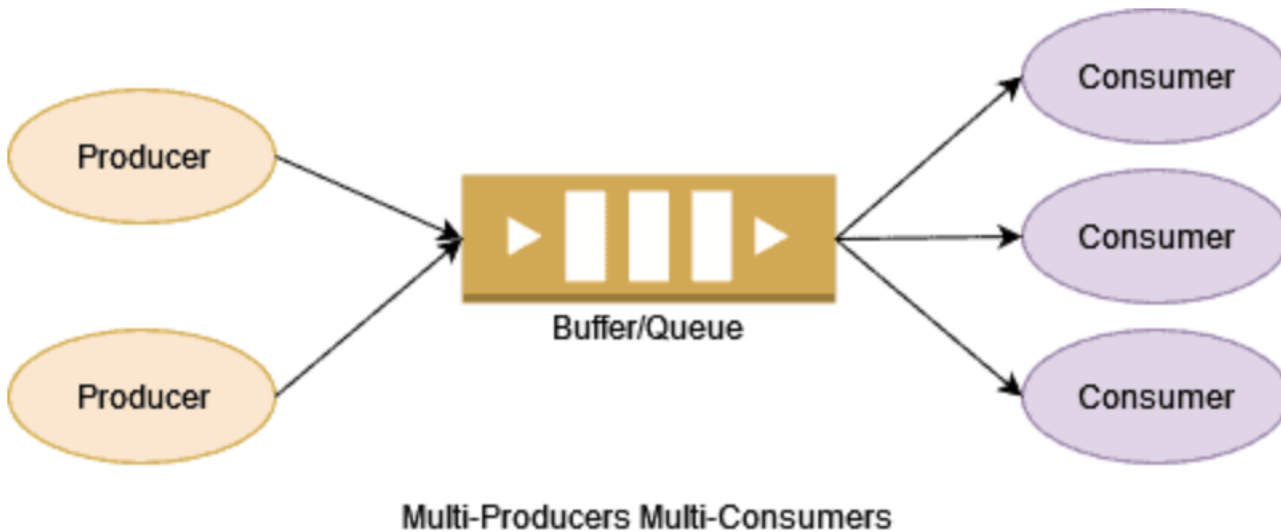
Producer Consumer Problem



- Both producer and consumer may try to update the queue at the same time. This could lead to data loss or inconsistencies.
- Producers might be slower than consumers. In such cases, the consumer would process elements fast and wait.
- In some cases, the consumer can be slower than the producer. This situation leads to a queue overflow issue.
- In real scenarios, we may have multiple producers, multiple consumers, or both. This may cause the same message to be processed by different consumers.

Producer Consumer Problem

- In real scenarios, we may have multiple producers, multiple consumers, or both. This may cause the same message to be processed by different consumers.



● Producer Consumer Demo

- • Run the code available on Brightspace.



Producer Consumer Explanation

- In PC class (A class that has both produce and consume methods), a linked list of jobs and a capacity of the list is added to check that producer does not produce if the list is full.
- In Producer class, the value is initialized as 0.
 - Also, we have an infinite outer loop to insert values in the list. Inside this loop, we have a synchronized block so that only a producer or a consumer thread runs at a time.
 - An inner loop is there before adding the jobs to list that checks if the job list is full, the producer thread gives up the intrinsic lock on PC and goes on the waiting state.
 - If the list is empty, the control passes to below the loop and it adds a value in the list.

Producer Consumer Explanation (cont.)

- In the Consumer class, we again have an infinite loop to extract a value from the list.
 - Inside, we also have an inner loop which checks if the list is empty.
 - If it is empty, then we make the consumer thread give up the lock on PC and passes the control to producer thread for producing more jobs.
 - If the list is not empty, we go round the loop and removes an item from the list.
- In both the methods, we use notify at the end of all statements. The reason is simple, once you have something in list, you can have the consumer thread consume it, or if you have consumed something, you can have the producer produce something.
- `sleep()` at the end of both methods just make the output of program run in step wise manner and not display everything all at once so that you can see what is happening in the program.



Deadlocks



Deadlocks

- Definition: A Deadlock occurs in a multithreaded environment when two or more threads are blocked forever, waiting for each other to release resources that they need to proceed. It's a situation where each thread holds a resource and waits for another resource held by a different thread, resulting in a stalemate.
- Conditions:
 - Deadlock typically arises due to four necessary conditions:
 - Mutual Exclusion: At least one resource must be held in a non-shareable mode, meaning only one thread can use it at a time.
 - Hold and Wait: A thread must hold at least one resource and be waiting to acquire additional resources held by other threads.
 - No Preemption: Resources cannot be forcibly taken from the threads holding them; they can only be released voluntarily.
 - Circular Wait: There must be a circular chain of two or more threads, each waiting for a resource held by the next thread in the chain.

● Deadlock Demo

- • Run the code available on Brightspace.



Deadlock Avoidance

- Avoiding circular waits by ensuring a total ordering of resources.
- Using timeouts to limit the duration of resource acquisition attempts.
- Employing deadlock detection and recovery mechanisms.
- Ensuring that locks are acquired in a consistent order across all threads.
- Minimizing the holding time for locks.



Lock and Condition Interfaces



Lock Interface

- The Lock interface provides a more flexible and powerful mechanism for controlling access to shared resources than synchronized blocks/methods.
- It allows explicit control over the locking and unlocking of critical sections of code.
- A lock is acquired using the `lock()` method and released using the `unlock()` method.
- It provides additional methods like `tryLock()` for non-blocking attempts to acquire the lock.



● Condition Interface

- The `Condition` interface provides a way for threads to suspend execution and wait for a specific condition to be met.
- It is associated with a `Lock` instance and is obtained using the `newCondition()` method.
- Threads waiting on a condition can be signaled by other threads using the `signal()` or `signalAll()` methods.
- Typically, conditions are used to manage waiting and signaling for specific resources or state changes.

● Lock and Condition Demo

- • Run the code available on Brightspace.



Part II Resources



University of Windsor



Useful Resources





Useful Resources

- <https://www.baeldung.com/java-producer-consumer-problem>
- <https://www.geeksforgeeks.org/multithreading-in-java/?ref=lbp>
- <https://www.geeksforgeeks.org/synchronization-in-java/?ref=lbp>
- <https://www.youtube.com/watch?v=4GmooU2R8lQ>



Thank you!
Questions?

Dr. Andreas S. Maniatis

Assistant Professor

Andreas.Maniatis@UWindsor.ca

<http://www.linkedin.com/in/andreasmaniatis>



University of Windsor